

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work No. 1:

Caesar Cipher

Elaborated:
Poiata Calin

gr. FAF-243

Verified:
Maia Zaica

Chișinău – 2026

Table of Contents

Short Theory	1
The Caesar Cipher	1
The Caesar Cipher with a Keyword Permutation	1
Implementation	2
Alphabet Encoding (Table 2)	2
Input Normalization	2
Input Validation	2
Encryption and Decryption (Task 1.1)	3
Permuted Alphabet and Two-Key Cipher (Task 1.2)	3
Results	4
Task 1.1: Standard Caesar Cipher	4
Task 1.2: Caesar Cipher with Permutation	6
Conclusions	8
Source Code	8

Short Theory

The Caesar Cipher

The Caesar cipher is one of the oldest and simplest known encryption techniques. Every letter of the plaintext is replaced by the letter found a fixed number of positions further along the alphabet. The secret numeric key k is the same for both encryption and decryption, and must satisfy $k \in \{1, 2, \dots, n-1\}$, where n is the size of the alphabet (the value $k = 0$ is excluded because it leaves the message unchanged).

Encryption and decryption formulas:

$$c = e_k(x) = (x + k) \bmod n$$

$$m = d_k(y) = (y - k) \bmod n$$

where x is the numeric code (position, 0-indexed) of a plaintext letter and y is the numeric code of a ciphertext letter. When $y - k$ is negative, n is added before applying modulo to keep the result in the range $[0, n-1]$.

For the Romanian alphabet ($n = 31$, Table 1) and key $k = 5$, letter **E** (code 6) encrypts to **J** (code 11): $(6 + 5) \bmod 31 = 11$.

Key space: there are $n - 1 = 30$ possible keys, which makes the cipher trivially breakable by exhaustive search (trying all keys in turn).

The Caesar Cipher with a Keyword Permutation

To increase the key space, the alphabet is first permuted using a second key — a keyword. The keyword letters (unique, in order of first appearance) are written first, followed by the remaining alphabet letters in their natural order. All 31 letters appear exactly once in the permuted alphabet.

The numeric code of each letter is then its **position in the permuted alphabet**, and the same shift formula is applied:

$$c = e_{k_1, k_2}(x) = (x' + k_1) \bmod n, \quad m = d_{k_1, k_2}(y) = (y' - k_1) \bmod n$$

where x' and y' are positions in the permuted alphabet.

Key space: there are $31! \times 30$ possible key pairs (k_1, k_2) , an astronomically larger space. However, the cipher remains vulnerable to **frequency analysis**, because every plaintext letter is always mapped to the same ciphertext letter.

Implementation

Alphabet Encoding (Table 2)

The Romanian alphabet ($n = 31$) is stored as a Python list, giving each letter an index from 0 to 30 without relying on ASCII or Unicode code points:

```
ROMANIAN_ALPHABET: List[str] = [
    'A', 'Ă', 'Ȃ', 'B', 'C', 'D', 'E', 'F', 'G', 'H',
    'I', 'Î', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q',
    'R', 'S', 'Ș', 'T', 'Ț', 'U', 'V', 'W', 'X', 'Y', 'Z'
]
ALPHABET_SIZE: int = len(ROMANIAN_ALPHABET)  # 31

CHAR_TO_INDEX: Dict[str, int] = {ch: i for i, ch in enumerate(ROMANIAN_ALPHABET)}
INDEX_TO_CHAR: Dict[int, str] = {i: ch for i, ch in enumerate(ROMANIAN_ALPHABET)}
```

Two dictionaries (`CHAR_TO_INDEX` and `INDEX_TO_CHAR`) provide $O(1)$ lookup in both directions. The shift arithmetic operates on dictionary indices — never on Unicode code points.

Input Normalization

Romanian text may arrive with **cedilla** variants (Ș, Ț) instead of **comma-below** variants (Ș̣, Ț̣), depending on the keyboard layout. A translation table converts them before any further processing:

```
CEDILLA_TRANS = str.maketrans({'Ș': 'Ș', 'ș': 'ș', 'Ț': 'Ț', 'ț': 'ț'})

def normalize_input_text(raw: str) → str:
    return unicodedata.normalize('NFC', raw).translate(CEDILLA_TRANS).upper()
```

`unicodedata.normalize('NFC', ...)` ensures that decomposed Unicode sequences (e.g. `s` + combining cedilla) are first composed into a single code point before the substitution map is applied.

Input Validation

Key Validation

```
def validate_key(key_str: str) → Tuple[bool, int, str]:
    try:
        val = int(key_str.strip())
    except ValueError:
        return False, 0, f"Invalid key '{key_str}'. Must be an integer between 1 and 30."
    if not (1 ≤ val ≤ 30):
        return False, 0, f"Key {val} out of range. Must be between 1 and 30 inclusive."
    return True, val, ""
```

The function returns a `(is_valid, value, error_message)` triple. The caller displays the error message and re-prompts until a valid value is entered.

Text Validation

```
def validate_and_clean_text(raw: str) → Tuple[bool, str, List[str], str]:
    normalized = normalize_input_text(raw)
    cleaned, rejected = [], []
    for orig, norm in zip(raw, normalized):
        if orig.isspace():
            continue  # spaces are allowed and stripped
        if norm in CHAR_TO_INDEX:
            cleaned.append(norm)
        elif orig not in rejected:
            rejected.append(orig)
    if rejected:
        return False, "", rejected, f"Invalid character(s): {rejected}"
    return True, "".join(cleaned), [], ""
```

Each character is checked against `CHAR_TO_INDEX`. Any character not in the Romanian alphabet (after normalization) is collected and reported to the user.

Keyword Validation

```
def validate_keyword(raw: str) → Tuple[bool, str, List[str], str]:
    normalized = normalize_input_text(raw.strip())
    cleaned, rejected = [], []
    for orig, norm in zip(raw.strip(), normalized):
        if norm in CHAR_TO_INDEX:
            cleaned.append(norm)
        elif orig not in rejected:
            rejected.append(orig)
    if rejected:
        return False, "", rejected, f"Invalid character(s) in keyword: {rejected}"
    kw = "".join(cleaned)
    if len(kw) < 7:
        return False, "", [], f"Keyword '{kw}' too short ({len(kw)}). Minimum length is 7."
    return True, kw, [], ""
```

Spaces in the keyword are treated as invalid characters (unlike plaintext). After removing bad characters the remaining length is checked against the minimum.

Encryption and Decryption (Task 1.1)

```
def caesar_encrypt(plaintext: str, key: int) → str:
    return "".join(
        INDEX_TO_CHAR[(CHAR_TO_INDEX[ch] + key) % ALPHABET_SIZE]
        for ch in plaintext
    )

def caesar_decrypt(ciphertext: str, key: int) → str:
    return "".join(
        INDEX_TO_CHAR[(CHAR_TO_INDEX[ch] - key + ALPHABET_SIZE) % ALPHABET_SIZE]
        for ch in ciphertext
    )
```

Adding `ALPHABET_SIZE` before the modulo in decryption avoids negative remainders, which some languages (Python included) can produce for negative operands of `%`.

Permuted Alphabet and Two-Key Cipher (Task 1.2)

```
def generate_permuted_alphabet(keyword: str) → List[str]:
    seen, permuted = set(), []
    for ch in keyword:
        # distinct keyword letters first
        if ch not in seen:
            seen.add(ch); permuted.append(ch)
    for ch in ROMANIAN_ALPHABET:
        # remaining letters in natural order
        if ch not in seen:
            seen.add(ch); permuted.append(ch)
    return permuted
    # always exactly 31 entries

def caesar_permuted_encrypt(plaintext, key, keyword):
    perm = generate_permuted_alphabet(keyword)
    c2i = {ch: i for i, ch in enumerate(perm)}
    i2c = {i: ch for i, ch in enumerate(perm)}
    return "".join(i2c[(c2i[ch] + key) % ALPHABET_SIZE] for ch in plaintext), perm

def caesar_permuted_decrypt(ciphertext, key, keyword):
    perm = generate_permuted_alphabet(keyword)
    c2i = {ch: i for i, ch in enumerate(perm)}
    i2c = {i: ch for i, ch in enumerate(perm)}
    return "".join(i2c[(c2i[ch] - key + ALPHABET_SIZE) % ALPHABET_SIZE] for ch in ciphertext), perm
```

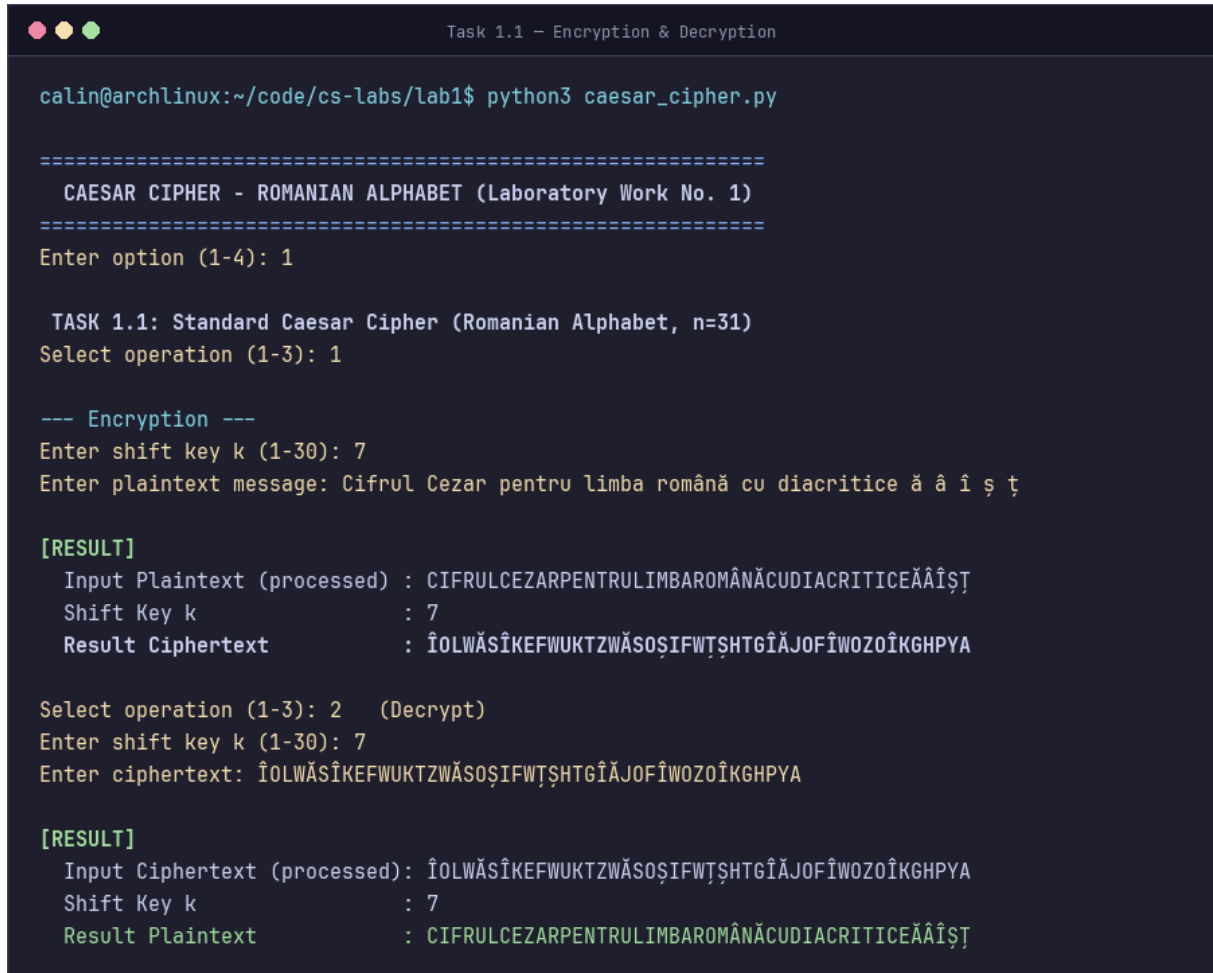
The permuted alphabet is rebuilt on every call from the same keyword, so encryption and decryption are perfectly symmetric.

Results

Task 1.1: Standard Caesar Cipher

Encryption and Decryption

The screenshot below shows a full encryption and decryption cycle with key $k = 7$. Input text containing Romanian diacritics is normalised, uppercased and stripped of spaces before the cipher is applied; decryption recovers the exact processed plaintext.



```
Task 1.1 - Encryption & Decryption

calin@archlinux:~/code/cs-labs/lab1$ python3 caesar_cipher.py

=====
  CAESAR CIPHER - ROMANIAN ALPHABET (Laboratory Work No. 1)
=====
Enter option (1-4): 1

TASK 1.1: Standard Caesar Cipher (Romanian Alphabet, n=31)
Select operation (1-3): 1

--- Encryption ---
Enter shift key k (1-30): 7
Enter plaintext message: Cifrul Cezar pentru limba română cu diacritice ă â î ș ț

[RESULT]
Input Plaintext (processed) : CIFRULCEZARPENTRULIMBAROMÂNĂCUDIACRITICEĂÂÎȘȚ
Shift Key k                  : 7
Result Ciphertext            : ÎOLWĂŜÎKEFWUKTZWĂSOȘIFWȚȘHTGÎĂJOFÎWOZOÎKGHPYA

Select operation (1-3): 2    (Decrypt)
Enter shift key k (1-30): 7
Enter ciphertext: ÎOLWĂŜÎKEFWUKTZWĂSOȘIFWȚȘHTGÎĂJOFÎWOZOÎKGHPYA

[RESULT]
Input Ciphertext (processed): ÎOLWĂŜÎKEFWUKTZWĂSOȘIFWȚȘHTGÎĂJOFÎWOZOÎKGHPYA
Shift Key k                  : 7
Result Plaintext             : CIFRULCEZARPENTRULIMBAROMÂNĂCUDIACRITICEĂÂÎȘȚ
```

Figure 1: Task 1.1 — encryption with $k = 7$ and subsequent decryption

Invalid Key Input

The program re-prompts for a key when the value is out of range (0 or 31) or non-numeric, displaying a clear message that states the allowed range [1, 30].

```
Task 1.1 - Invalid Key Validation

calin@archlinux:~/code/cs-labs/lab1$ python3 caesar_cipher.py
Enter option (1-4): 1 | Select operation (1-3): 1

--- Encryption ---
Enter shift key k (1-30): 0
[ERROR] Key value 0 is out of range. Allowed: integer between 1 and 30 inclusive.
Enter shift key k (1-30): 31
[ERROR] Key value 31 is out of range. Allowed: integer between 1 and 30 inclusive.
Enter shift key k (1-30): -5
[ERROR] Key value -5 is out of range. Allowed: integer between 1 and 30 inclusive.
Enter shift key k (1-30): abc
[ERROR] Invalid key 'abc'. The shift key must be an integer between 1 and 30 inclusive.
Enter shift key k (1-30): 15 ✓
Enter plaintext message: TEST REUSIT

[RESULT]
Input Plaintext (processed) : TESTREUSIT
Shift Key k                  : 15
Result Ciphertext           : FSDFCSHDUF
```

Figure 2: Task 1.1 — invalid key handling (out-of-range and non-numeric values)

Invalid Characters and Cedilla Normalization

Digits, punctuation and other non-Romanian characters are rejected with a message listing the offending symbols. The cedilla variants Ș/ș and Ț/ț are silently normalised to the correct comma-below forms Ș̣/ș̣ and Ț̣/ț̣.

```
Task 1.1 - Invalid Character & Cedilla Normalization

calin@archlinux:~/code/cs-labs/lab1$ python3 caesar_cipher.py
Enter shift key k (1-30): 3
Enter plaintext message: Salutare! Anul 2026 aduce securitate 100%.
[ERROR] Invalid character(s) rejected: '!', '2', '0', '6', '1', '%', '.'.
Allowed: Romanian letters (A-Z, Ă, Â, Î, Ș, Ț, case-insensitive) and spaces.
Enter plaintext message: Ștefan cel Mare și Sfânt ← cedilla variant (Ș/ș)

[RESULT] (cedilla letters Ș → Ș̣ auto-normalized before processing)
Input Plaintext (processed) : Ș̣TEFANCELMAREȘ̣ISFÂNT
Shift Key k                  : 3
Result Ciphertext           : UVHIBQFHOPBTHUKȚ̣IDQV
```

Figure 3: Task 1.1 — invalid character rejection and automatic cedilla normalization

Task 1.2: Caesar Cipher with Permutation

Permuted Alphabet, Encryption and Decryption

With keyword **CRİPTOGRAFİE** and shift $k_1 = 12$ the program first displays the 31-letter permuted alphabet (for manual verification), then encrypts the plaintext and subsequently decrypts the ciphertext back to the original.

```
Task 1.2 - Permuted Alphabet, Encryption & Decryption

calin@archlinux:~/code/cs-labs/lab1$ python3 caesar_cipher.py
Enter option (1-4): 2
TASK 1.2: Caesar Cipher with Permutation (Two Keys, n=31)
Select operation (1-3): 1 (Encrypt)

Enter Key 1 (shift k, 1-30): 12
Enter Key 2 (keyword, ≥7 Romanian letters): CRİPTOGRAFİE
Enter plaintext message: Securitatea Informației și Tehnologia Calculatoarelor

=== Permuted Alphabet (Keyword: 'CRİPTOGRAFİE') (n = 31) ===
Index:  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
Letter:  C  R  I  P  T  O  G  A  F  E  Ă  Â  B  D  H  Î
-----
Index: 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
Letter:  J  K  L  M  N  Q  S  Ș  Ț  U  V  W  X  Y  Z

[RESULT]
Plaintext  : SECURITATEAINFORMAȚIEIȘITEHNOLOGIACALCULATOARELOR
Key 1 (k)  : 12      Key 2 (keyword): CRİPTOGRAFİE
Ciphertext : PQBGDHJMJQMHRNKDCMOHQHHTJQVRKZKLHMBMZBGZMJKMDQZKD

Select operation (1-3): 2 (Decrypt)
Enter Key 1 (shift k, 1-30): 12
Enter Key 2 (keyword, ≥7 Romanian letters): CRİPTOGRAFİE
Enter ciphertext: PQBGDHJMJQMHRNKDCMOHQHHTJQVRKZKLHMBMZBGZMJKMDQZKD
[RESULT]
Result Plaintext : SECURITATEAINFORMAȚIEIȘITEHNOLOGIACALCULATOARELOR
```

Figure 4: Task 1.2 — permuted alphabet display, encryption and decryption with $k_1 = 12$, keyword CRİPTOGRAFİE

Invalid Keyword Input

The program rejects keywords that are too short (fewer than 7 letters), contain spaces, digits or punctuation, and continues prompting until a valid keyword is given.

```
Task 1.2 – Keyword Validation (Length & Character Errors)

calin@archlinux:~/code/cs-labs/lab1$ python3 caesar_cipher.py
Enter option (1-4): 2      |      Select operation (1-3): 1
Enter Key 1 (shift k, 1-30): 5

Enter Key 2 (keyword, ≥7 Romanian letters): CEZAR
[ERROR] Keyword 'CEZAR' is too short (length 5). Minimum length is 7 characters.
Enter Key 2 (keyword, ≥7 Romanian letters): CRIPTO 123
[ERROR] Invalid character(s) in keyword rejected: ' ', '1', '2', '3'.
The keyword must contain only Romanian alphabet letters (A-Z, Ă, Â, Î, Ș, Ț).
Enter Key 2 (keyword, ≥7 Romanian letters): ALGORITM!
[ERROR] Invalid character(s) in keyword rejected: '!'.
The keyword must contain only Romanian alphabet letters (A-Z, Ă, Â, Î, Ș, Ț).
Enter Key 2 (keyword, ≥7 Romanian letters): CRIPTOGRAFIE  ✓
Enter plaintext message: ALFABET PERMUTAT VALID

[RESULT]
Ciphertext: BȘDBKHEFHGȚZEBECBȘAL
```

Figure 5: Task 1.2 — keyword validation: too short, digits, and punctuation errors

Conclusions

Security of the standard Caesar cipher. With only $n - 1 = 30$ possible keys for the Romanian alphabet, an attacker can recover the plaintext in at most 30 trials — a trivially fast exhaustive search. The cipher provides no meaningful security and should never be used to protect real information.

Security of the permuted Caesar cipher. Adding a keyword-based permutation inflates the key space to $31! \times 30 \approx 10^{33}$ possible key pairs, making brute-force enumeration completely infeasible. Despite this, the cipher remains vulnerable to **frequency analysis**: because each plaintext letter is always replaced by the same ciphertext letter, the statistical structure of the Romanian language (letter frequencies, common digraphs, etc.) is preserved in the ciphertext and can be exploited to recover the plaintext without knowing the key.

What was learned.

- How to map an arbitrary alphabet to integer codes and apply modular arithmetic for encryption and decryption, without relying on built-in character encodings.
- How a keyword permutation is constructed and how it extends the classical Caesar cipher.
- The practical limitations of both ciphers: small key space (Caesar) and susceptibility to frequency analysis (both variants), which are the fundamental reasons why modern symmetric ciphers use much more complex transformations.
- Implementation considerations for real-world Romanian text: Unicode normalization, NFC composition, and handling of the two keyboard variants for Ș/Ț.

Source Code

The full source code for this laboratory work is available on GitHub:

<https://github.com/colinnnnnnnnnnn/cs-labs>

The repository contains:

- `caesar_cipher.py` — core implementation and interactive CLI (Tasks 1.1 and 1.2)
- `test_caesar.py` — unit test suite (8 tests, all passing)
- `demo.py` — non-interactive demonstration script
- `generate_screenshots.py` — script used to render the terminal screenshots in this report